

Dati elementari, operatori

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



1 Variabili e Tipi di Dato

2 Costanti e Casting

3 Operatori e Precedenza

1 Variabili e Tipi di Dato

2 Costanti e Casting

3 Operatori e Precedenza

Le Variabili in C e Dichiarazione

Che cos'è una variabile?

Una **variabile** è una locazione di memoria con un nome che memorizza un valore modificabile durante l'esecuzione.

Sintassi

```
tipo nome_variabile; // Dichiarazione  
tipo nome_variabile = valore_iniziale; // Inizializzazione
```

Evoluzione delle Regole di Dichiarazione

- **C89:** Le variabili **devono** essere dichiarate tassativamente all'inizio del blocco/funzione.
- **C99 e successivi:** È possibile dichiarare le variabili in qualsiasi punto del codice.
- **Buone pratiche:** Dichiarare all'inizio le variabili principali della funzione; dichiarare variabili locali/temporanee (es. indici di cicli) direttamente nel blocco dove servono.

Che cos'è un Tipo di Dato?

Specifica la natura dei dati memorizzati, la **dimensione in memoria** allocata e le **operazioni consentite**.

Classificazione delle 32 Keyword (C89)

Tipi di Dato Fondamentali

char, int, float, double, void, short,
long, signed, unsigned

Controllo del Flusso

if, else, switch, case, default, for,
while, do, break, continue, goto, return

Classi di Storage

auto, register, static, extern

Tipi Definiti dall'Utente

struct, union, enum, typedef

Qualificatori e sizeof

const, volatile, sizeof

Tipi Primitivi e Modificatori

Tipo Base	Specif.	Min. Garantito*	Descrizione
char	%c	8 bit (1 Byte)	Carattere o piccolo intero.
short int	%hd	16 bit	Intero con intervallo ridotto.
int	%d	16 bit (spesso 32)	Intero standard.
long int	%ld	32 bit	Intero ad ampiezza estesa.
long long int	%lld	64 bit	Intero grande (dal C99).
float	%f	32 bit	Reale a singola precisione.
double	%lf	64 bit	Reale a doppia precisione.
void	-	0 bit	Assenza di tipo o valore.

Modificatori di segno

`signed` (con segno, default) oppure `unsigned` (solo valori ≥ 0 , raddoppia il valore massimo positivo).

Nota sulla dimensione in memoria

*La dimensione esatta dipende dall'architettura del sistema (32/64 bit) e dal compilatore. Per conoscere la dimensione esatta si usa l'operatore `sizeof()`.

Tipi Derivati

Derivano dai tipi base per gestire collezioni di dati o indirizzi di memoria:

- **Array**: sequenza omogenea di elementi contigui in memoria.
- **Puntatori**: variabili che memorizzano l'indirizzo di memoria di un'altra variabile.
- **Funzioni**: blocchi di codice con un tipo di ritorno.

Tipi Definiti dall'Utente

Permettono di estendere il linguaggio per rappresentare strutture dati complesse:

- **struct**: raggruppa variabili di tipo diverso sotto un unico nome.
- **union**: condivide la stessa memoria tra più membri.
- **enum**: definisce costanti con nome associato.
- **typedef**: crea un alias per un tipo di dato esistente.

Interi a Dimensione Fissa (C99)

Tipi ad Ampiezza Esatta (<stdint.h>)

Poiché la dimensione di `int` dipende dall'architettura, C99 introduce tipi a dimensione garantita:

`int8_t` / `uint8_t` Interi a 8 bit con o senza segno.

`int16_t` / `uint16_t` Interi a 16 bit con o senza segno.

`int32_t` / `uint32_t` Interi a 32 bit con o senza segno.

`int64_t` / `uint64_t` Interi a 64 bit con o senza segno.

`intptr_t` Intero garantito per contenere un indirizzo di memoria (puntatore).

Costanti di Limite (<limits.h>)

Definiscono i limiti numerici del sistema: `INT_MIN`, `INT_MAX`, `SHRT_MIN`, `UINT16_MAX`, ecc.

Reali e Complessi (C99)

- `long double`: precisione estesa.
- `_Complex`: gestione numeri complessi (es. `double _Complex z`).

Limiti Reali (<float.h>)

- `FLT_EPSILON`: min x t.c. $1.0 + x > 1.0$.
- `FLT_MIN` / `FLT_MAX`: range min/max normalizzato per i float.

Tipi Booleani (C99/C11)

Nel C tradizionale non esiste un tipo booleano:

- 0 equivale a **false**.
- Qualsiasi valore $\neq 0$ è **true**.

Con `<stdbool.h>` (C99+):

- Tipo nativo: `_Bool` / alias `bool`.
- Parole chiave: `true` (1) e `false` (0).

1 Variabili e Tipi di Dato

2 Costanti e Casting

3 Operatori e Precedenza

Definizione

Le **costanti** (o **literals**) sono valori fissi non modificabili durante l'esecuzione.

Caratteristica	const	#define
Natura	Variabile a sola lettura	Direttiva del preprocessore
Controllo tipo	Sì (verificato dal compilatore)	No (sostituzione testuale)
Ambito (Scope)	Rispetta le regole del blocco	Valido dal punto di definiz. a fine file
Debugging	Ispezionabile da debugger	Non visibile nel debugger

Tipi di Literals

- **Interi:** 42, 0x2F (esadecimale), 052 (ottale), 100L (long).
- **Floating-point:** 3.14, 1.5e-3, 2.0f (float).
- **Carattere / Stringa:** 'A', '\n', "Hello, World!".

Conversione di Tipo (Casting)

1. Conversione Implicita (Promozione di Tipo)

Eseguita automaticamente per evitare perdite di precisione in espressioni miste:

`char / short → int → unsigned int → long → float → double`

2. Casting Esplicito

Forzato con la sintassi: `(tipo)`
espressione

```
int somma = 15, c = 2;  
// Divisione float forzata (7.5)  
double m = (double) somma / c;
```

Rischio: Troncamento

La conversione verso tipi con precisione minore taglia le cifre decimali:

```
double pi = 3.99;  
int x = (int) pi; // x diventa 3!
```

1 Variabili e Tipi di Dato

2 Costanti e Casting

3 Operatori e Precedenza

Operatori

Sulle variabili agiscono degli *operatori* per costruire delle *espressioni*:

Operatore di assegnamento: `=`

Operatori aritmetici: gli usuali `+` `-` `*` `/`

Operatori aritmetici bitwise: `<<` `>>`

Operatori booleani (bitwise): `|` `&` `~` `^`

Operatori relazionali: `<` `>` `==` `!=`

Operatori di incremento: `++` `--`

Operatori logici: `||` `&&`

Ogni espressione ha un valore; inoltre, gli operatori hanno una precedenza di associazione (ma si possono usare le parentesi).

Operatori Aritmetici e di Assegnamento

1. Operatori Aritmetici

- $+$, $-$, $*$, $/$: Somma, sottrazione, prodotto, divisione.
- $\%$ (**Modulo**): Resto della divisione intera (es. $7 \% 3 \rightarrow 1$). *Valido solo su interi!*

2. Assegnamento Composto

Sintassi compatta per aggiornare variabili:

- $a += b \rightarrow a = a + b$
- Disponibili anche: $-=$, $*=$, $/=$, $\%=$

3. Incremento e Decremento

Modificano il valore di 1: $++$ e $--$.

Prefisso vs Postfisso:

- $++x$ (*Pre*): incrementa **prima** di usare il valore nell'espressione.
- $x++$ (*Post*): usa il valore attuale e incrementa **dopo**.

Il Problema

In C, la divisione tra due interi genera **sempre un intero** (con troncamento della parte decimale), anche se il risultato viene assegnato a una variabile `double`!

Attenzione agli operatori interi:

```
1  int main(int argc, char *argv[]) {  
2      double result = 1 / 2 * 2;  
3      printf("Result is %lf\n", result);  
4      return 0;  
5  }
```

Il Problema

In C, la divisione tra due interi genera **sempre un intero** (con troncamento della parte decimale), anche se il risultato viene assegnato a una variabile double!

Attenzione agli operatori interi:

```
1  int main(int argc, char *argv[]) {  
2      double result = 1 / 2 * 2;  
3      printf("Result is %lf\n", result);  
4      return 0;  
5  }
```

Operatore di *cast*:

```
1  int main(int argc, char *argv[]) {  
2      double result = ((double) 1) / 2 * 2;  
3      printf("Result is %lf\n", result);  
4      return 0;  
5  }
```

Operatori Relazionali

Confrontano valori e restituiscono 1 (vero) o 0 (falso):

- == (Uguaglianza)
- != (Diversità)
- >, <, >=, <=

Attenzione! Non confondere = (assegnamento) con == (confronto).

Operatori Logici

Combinano espressioni booleane:

- && (**AND**): Vero solo se entrambi gli operandi sono veri.
- || (**OR**): Vero se almeno un operando è vero.
- ! (**NOT**): Inverte il valore di verità.

Operatori Bitwise (Bit a Bit)

Operano direttamente sulla rappresentazione binaria dei dati interi:

- $\&$: AND bitwise
- $|$: OR bitwise
- $\wedge$: XOR bitwise (OR esclusivo)
- $\sim$: NOT bitwise (complemento a 1)
- $\ll$: Shift a sinistra (moltiplica per 2^n)
- $\gg$: Shift a destra (divide per 2^n)

Come vengono valutate le espressioni?

Quando più operatori compaiono nella stessa espressione, l'ordine di valutazione dipende dalla loro **precedenza** e dalla loro **associatività**.

Priorità	Operatori	Associatività
1 (<i>Massima</i>)	() [] -> .	Da sinistra a destra
2	! ~ ++ - + - (unari) (type) sizeof & *	Da destra a sinistra
3	* / %	Da sinistra a destra
4	+ -	Da sinistra a destra
5	< >	Da sinistra a destra
6	< <= > >=	Da sinistra a destra
7	== !=	Da sinistra a destra
8	&&	Da sinistra a destra
9		Da sinistra a destra
10	? : (Ternario)	Da destra a sinistra
11 (<i>Minima</i>)	= += -= *= /= %= ecc.	Da destra a sinistra

Golden Rule

In caso di dubbio, **usare le parentesi tonde** $()$ per esplicitare l'ordine desiderato e migliorare la leggibilità.